

Major Project Documentation(Report): Cosmic Compass: An Agentic RAG SME

Team KnowlAgents

Gargi Shroff(2022114009), Ameya Rathod(2022111021)

November 16, 2025

Contents

1	Project Overview and Objective	2
2	System Architecture	2
2.1	Directory Structure	2
2.2	Architectural Flow	3
3	Core Component Implementation	4
3.1	Data Ingestion & Processing (data_processing/process.py)	4
3.2	Embedding & Indexing (rag/indexer.py)	5
3.3	Hybrid Retrieval (rag/rag_tools.py)	5
3.4	Agentic Workflow (core/agent_self2.py)	5
3.5	Middleware & Guardrails (core/agent_self2.py)	6
3.6	Custom Tools (tools/sme_tools.py)	6
4	Setup and Run Instructions	6
4.1	Prerequisites	6
4.2	Step-by-Step Installation	7
4.3	Data Processing and Indexing	7
4.4	Running the Application	8
5	Evaluation Criteria Self-Assessment	8
5.1	Domain Definition & SME Design	8
5.2	Data Collection & Organization	8
5.3	Preprocessing & Chunking	8
5.4	Embeddings & Indexing	9
5.5	RAG Retrieval	9
5.6	SME Capabilities	9
5.7	Agent Workflow & Tools	9
5.8	Prompting Strategy	9
5.8.1	Prompting Strategy & Iterations	9
5.9	Document Generation & Email	10
5.10	System Architecture & APIs	11
5.11	Final Deliverables & Demo	11

5.12 Bonus	11
6 Conclusion and Future Work	11
7 Demo Video	11

1 Project Overview and Objective

This project implements **Cosmic Compass**, a highly extensible, multi-agent Retrieval-Augmented Generation (RAG) system designed to act as a Subject Matter Expert (SME) in the domain of **Astronomy and Cosmology**.

The primary objective is to fulfill complex, multi-step user requests by integrating several key technologies:

- **Agentic Workflows:** A supervisor-worker agent architecture built with LangChain and LangGraph for planning, routing, and tool execution.
- **Multi-Modal RAG:** A sophisticated retrieval system that handles both text and images, combining keyword (BM25) and semantic (CLIP) search over a corpus of astronomy documents.
- **Tool-Using Capabilities:** A suite of custom tools for generating documents (.pdf, .docx) and sending email notifications.
- **Human-in-the-Loop (HITL):** An advanced verification system that pauses the agent's execution to ask the user for approval before performing sensitive actions, such as generating a document or sending an email.

This system is exposed via a FastAPI backend and a modern, interactive web interface.

2 System Architecture

The system is designed with a modular, decoupled architecture, separating concerns into distinct directories. This design ensures maintainability and scalability.

2.1 Directory Structure

The project is organized as follows:

- **api/:** Contains the FastAPI backend (`main2.py`) and the HTML frontend (`index2.html`). This is the user-facing layer.
- **core/:** Contains the "brains" of the system, defining the agents (`agent_self2.py`), their prompts, and their middleware.
- **data_processing/:** Holds the data ingestion script (`process.py`) responsible for reading and chunking source documents.
- **docs/:** The raw source documents (PDFs, etc.) that form the SME's knowledge base.



Figure 1: Project Directory Structure.

- **rag/:** Contains all components for the RAG pipeline: the `indexer.py` script and the `rag_tools.py` (which defines the retriever).
- **tools/:** Contains the Python definitions for all custom tools the agents can use, such as `sme_tools.py`.
- **outputs/:** The destination for processed data (chunks, images, metadata) and generated reports.

2.2 Architectural Flow

The system operates on a request-response cycle designed for Human-in-the-Loop verification:

1. **Frontend (Login):** User enters their email (as `thread_id`) and optional format preference on `index2.html`.
2. **Frontend (Chat):** User sends a message. The frontend makes a POST request to `/api/chat` with the full message history and `thread_id`.
3. **Backend (main2.py):** The FastAPI server receives the request. It invokes the `supervisor_agent` using the provided `thread_id`.

4. **Core (agent_self2.py):** The `supervisor_agent` plans the task. It attempts to call a tool (e.g., `content_generator...`).
5. **HITL Interrupt:** The `HumanInTheLoopMiddleware` intercepts this call. The agent's state is paused (using the `InMemorySaver`), and the `invoke` call returns an `__interrupt__` message.
6. **Backend Response:** `main2.py` serializes the interrupt data and sends it back to the frontend as a JSON response (e.g., `{"type": "interrupt", ...}`).
7. **Frontend (Modal):** The JavaScript receives the "interrupt" message and displays a pop-up modal, showing the tool and its arguments. The user can "Approve," "Reject," or "Edit" the arguments.
8. **Frontend (Resume):** The user's decision is sent to a *new* endpoint: `/api/chat/resume`.
9. **Backend (Resume):** `main2.py` receives the decision, creates a `LangGraph Command`, and invokes the agent *again* on the same `thread_id`.
10. **Core (Resume):** The agent resumes from its saved checkpoint, processes the user's decision (e.g., runs the approved tool), and continues until it's finished or hits another interrupt.
11. **Final Response:** Once the workflow is complete, the supervisor agent returns its final message, which the backend sends to the frontend as a `{"type": "message", ...}` response.

3 Core Component Implementation

3.1 Data Ingestion & Processing (`data_processing/process.py`)

This script is the first step in the RAG pipeline.

- **Multi-Modal Extraction:** It handles `.pdf` (using `PyMuPDF/fitz`), `.docx` (using `docx2txt`), `.pptx` (using `python-pptx`), `.md`, and `.txt` files.
- **Image Handling:** It extracts images from PDFs, DOCX, and PPTX files, saving them to `outputs/images/` and inserting a text placeholder (e.g., `[Image: ...]`) in the extracted text.
- **Chunking Strategy:** It uses a content-aware, hierarchical chunking strategy. The text is first split by paragraphs. Long paragraphs are split into smaller "child chunks" (`CHILD_CHUNK_TOKENS = 256`). These child chunks are then re-combined into larger "parent chunks" (`PARENT_CHUNK_TOKENS = 1024`) to maintain context.
- **Metadata Manifest:** For each processed document, it generates a `.json` file in `outputs/metadata/`, which lists all the image and chunk files associated with that document.

3.2 Embedding & Indexing (rag/indexer.py)

This script reads the metadata manifests and populates the Elasticsearch database.

- **Vector Database:** Uses Elasticsearch (running on `http://localhost:9200`).
- **Index Mapping:** Creates an index named `rag-hybrid-index-v2` with fields for text content (`content`), vector embeddings (`embedding`), and metadata (`type`, `chunk_type`, `image_path`).
- **Multi-Modal Embeddings:** It uses the `clip-ViT-B-32` model from `sentence-transformers`. This is a crucial design choice, as this model can create meaningful vector embeddings for both text chunks and raw images.
- **Indexing Process:** It iterates through the metadata, embedding and indexing text chunks (with `type="text"`) and images (with `type="image"`).

3.3 Hybrid Retrieval (rag/rag_tools.py)

This file defines the core `HybridRetriever` logic, which is exposed to the agent as `rag_retriever_tool`.

- **Hybrid Search:** It combines two search methods for optimal relevance:
 1. **Keyword Search (BM25):** Elasticsearch performs a traditional text search.
 2. **Semantic Search (k-NN):** It performs a k-Nearest Neighbor search on the `clip-ViT-B-32` vector embeddings.
- **Reranking (Bonus):** The text results from *both* searches are combined, deduplicated, and then fed into a `CrossEncoder` (`ms-marco-MiniLM-L-6-v2`) for a more accurate reranking step.
- **Multi-Modal Logic:** The retriever logic explicitly separates text and image results. It includes a `min_images` parameter to perform a dedicated, image-only k-NN search if the initial query doesn't return enough images.

3.4 Agentic Workflow (core/agent_self2.py)

This is the "brain" of the system, defining the agents, their roles, and their interactions.

- **supervisor_agent:** The orchestrator. Its prompt explicitly tells it to plan, delegate, and chain tasks. Its "tools" are not simple Python functions, but wrappers for the other two agents. It first creates a To-do list for better planning and reasoning about the task.
- **content_generator_filtering_agent:** The specialist RAG agent. Its prompt instructs it on how to call the RAG tool, how to handle images, and how to format output for PDF (LaTeX) and DOCX (plain text).
- **email_agent:** The specialist utility agent. Its prompt strictly limits its role to calling the `send_email` tool.

- **Prompting Strategy:** The prompts are highly detailed and act as the core logic for the agents. They define constraints (e.g., "FORBIDDEN from using any other tool..."), workflows (e.g., "Step A... Step B..."), and error handling ("If the `content_generator` fails, STOP...").

3.5 Middleware & Guardrails (`core/agent_self2.py`)

The agents are enhanced with several middleware components:

- **HumanInTheLoopMiddleware:** This is the core of the UI. It is applied to the `supervisor_agent` and configured to **interrupt** execution before calling `content_generator...` or `email_agent...`. This allows the user to approve, reject, or edit the tool call.
- **ContentFilterMiddleware:** A custom guardrail that blocks requests containing banned keywords (e.g., "bombs," "drugs").
- **PIIMiddleware:** Masks or blocks sensitive information like credit card numbers and API keys.
- **ToolRetryMiddleware:** Automatically retries a failed tool call (e.g., network error) up to 3 times.

3.6 Custom Tools (`tools/sme_tools.py`)

These are the physical functions the agents can execute.

- **compile_latex_to_pdf:** Takes LaTeX code and a list of image paths. It creates a temporary directory, writes the `.tex` file, copies all required images into that directory, and runs `pdflatex` as a subprocess. This correctly handles image dependencies.
- **create_docx_report:** Uses `python-docx` to generate a `.docx` file and includes logic to read image bytes and insert them into the document.
- **send_email:** Uses `smtplib` to send an email via a Gmail SMTP server, with support for multiple attachments.

4 Setup and Run Instructions

To run the Cosmic Compass system, follow these steps.

4.1 Prerequisites

1. Python 3.10+ 2. A running Elasticsearch instance (e.g., via Docker). 3. A working `pdflatex` installation (e.g., TeX Live or MiKTeX). 4. Git (for cloning).

4.2 Step-by-Step Installation

1. Clone the Repository:

```
git clone <your-repo-url>
cd <your-repo-name>
```

2. Create & Activate Virtual Environment:

```
python3 -m venv venv
source venv/bin/activate
```

3. Install Dependencies: (Note: A requirements.txt file should be created)

```
pip install -r requirements.txt
```

4. Set Up Environment Variables: Create a file named .env in the project root directory. Add your API keys:

```
OPENROUTER_API_KEY="your-deepseek-api-key"
SMTP_API="your-gmail-app-password"
```

5. Verify Services: Ensure your Elasticsearch server is running and accessible at <http://localhost:9200>.

4.3 Data Processing and Indexing

You must run these steps from the root directory using Python's module flag (-m).

1. Process Documents: Run this command for *each* document in your docs/ folder.

```
python -m data_processing.process docs/astronomy_openstax.pdf
python -m data_processing.process docs/astronomy_for_mere_mortals.
pdf
... (etc. for all source files)
```

This will populate the outputs/chunks/, outputs/images/, and outputs/metadata/ directories.

2. Run the Indexer: This command reads the metadata and populates Elasticsearch.

```
python -m rag.indexer
```

You only need to do this once after processing your documents.

4.4 Running the Application

1. Start the FastAPI Server:

```
uvicorn api.main2:app --reload --port 8000
```

You will see a confirmation that the server is running on `http://localhost:8000`.

2. Access the UI: Open your web browser and navigate to `http://localhost:8000`.

3. Use the Application:

- (a) Enter your email (this is your unique chat ID) and optional format.
- (b) Send a message (e.g., "Create a quiz on black holes and email it to me").
- (c) Wait for the interrupt modal to appear.
- (d) Review the agent's plan (e.g., the arguments for `content_generator...`).
- (e) Click "Approve."
- (f) The agent will resume, generate the file, and then be interrupted **again** for the email.
- (g) Review the email details (recipient, attachment path).
- (h) Click "Approve" to send the email.

5 Evaluation Criteria Self-Assessment

This section explicitly addresses each of the project's evaluation criteria.

5.1 Domain Definition & SME Design

The domain is clearly defined as "Astronomy and Cosmology." The SME, "Cosmic Compass," has its capabilities (Q&A, report/quiz generation, email) explicitly defined in the prompts of the `supervisor_agent` and `content_generator...` in `core/agent_self2.py`.

5.2 Data Collection & Organization

The `docs/` directory contains a high-quality, relevant corpus of astronomy textbooks. The system is designed to handle heterogeneous formats (PDF, DOCX, PPTX, MD), as seen in `data_processing/process.py`. The metadata-driven approach (saving `.json` manifests) is a robust and well-justified organizational strategy.

5.3 Preprocessing & Chunking

The `data_processing/process.py` script implements a sophisticated, multi-stage pre-processing pipeline. This includes cleaning (`preprocess_text`) and a well-reasoned hierarchical chunking strategy (256-token child chunks nested in 1024-token parent chunks) to balance detail with context.

5.4 Embeddings & Indexing

The choice of `clip-ViT-B-32` in `rag/indexer.py` is a key strength, enabling multi-modal embedding of both text and images. The Elasticsearch index is correctly mapped for hybrid search (dense vector and text).

5.5 RAG Retrieval

The `HybridRetriever` in `rag/rag_tools.py` fully implements the hybrid search requirement (BM25 + k-NN). It also implements the `**bonus**` reranking step with a `CrossEncoder`. The `min_images` parameter shows advanced, goal-oriented retrieval logic.

5.6 SME Capabilities

The system provides high-accuracy Q&A via the RAG pipeline. The `content_generator...` agent demonstrates multi-step reasoning to synthesize retrieved context and generate high-quality quizzes and reports.

5.7 Agent Workflow & Tools

This is a core strength. The `supervisor_agent` in `core/agent_self2.py` is a complete implementation of "conversational planning, reasoning, [and] agent routing." The system implements complex multi-step workflows (e.g., generate → chain → email) and uses `InMemorySaver` for memory. The implementation of `HumanInTheLoopMiddleware` is an advanced form of tool-use logic.

5.8 Prompting Strategy

The system relies on advanced, structured system prompts in `core/agent_self2.py`. These prompts define roles, constraints, multi-step workflows (e.g., the LaTeX formatting rules), and error-handling logic, demonstrating a mastery of prompt-based learning.

5.8.1 Prompting Strategy & Iterations

This section addresses the evaluation criterion regarding the recording of observations and iterations to refine prompt design and system performance. The final, robust system prompts in `core/agent_self2.py` are the result of several iterative cycles of testing and refinement.

Initial Failures and Key Learnings Early iterations of the system, which used simpler, less-structured prompts and models, revealed several critical failure points:

- **Model Performance and Consistency:** Initial tests were conducted with smaller, open-source models. These models struggled with the multi-step reasoning and strict instruction-following required by the supervisor agent's prompt. Performance was inconsistent, and the agents often failed at complex chaining tasks. This led to the decision to fixate on the `deepseek/deepseek-chat` model, which provided consistently high-quality results across both complex content generation (like LaTeX) and agentic routing.

- **Failure of Naive Prompting:** The first version of the supervisor prompt was unstructured (e.g., "You are a helpful assistant that can generate reports and send emails"). This naive, zero-shot approach led to highly unreliable behavior. The agent would frequently "forget" the second half of a multi-step request; for example, it would generate a report but then fail to call the email agent. The generated output was also poor, as the agent didn't know *how* to format content for the specific tools.

Successful Strategies and Refinements The current system's reliability is built on the following successful strategies, which directly address the initial failures:

- **Strategy 1: Structured Workflow Prompting:** The supervisor's prompt was rewritten to be an explicit, step-by-step algorithm (e.g., "1. Plan...", "2. Delegate Step 1...", "3. Inspect & Chain..."). This declarative, "chain-of-thought" style prompt proved far more effective for multi-step reasoning than a simple role description.
- **Strategy 2: Proactive Memory Management:** To solve the "forgotten email" problem, the `index2.html` login screen was implemented. This captures the user's email and format preference *before* the chat begins, adding them to the agent's memory as a system message (e.g., "User email: ...", "User's preferred format is pdf"). The agent can then reliably access this information at any point in the workflow.
- **Strategy 3: Context-Specific Prompting (Zero-shot vs. One-shot):** We observed that different sub-agents required different prompting strategies for optimal performance:
 - **For DOCX Generation (Zero-shot):** The task of generating plain text for the `create_docx_report` tool was simple. A "zero-shot" instruction (e.g., "The `content` *must* be *plain text with NO Markdown*...") was sufficient to get reliable, unformatted text.
 - **For Beamer/PDF Generation (One-shot):** In contrast, generating LaTeX (especially Beamer slides) is a complex, syntax-heavy task. A zero-shot prompt was highly unreliable. We achieved consistent success by providing a complete, *one-shot example* (the two-slide Beamer template) directly in the `content_generator...` agent's prompt. This example acts as a concrete template, guiding the LLM to produce valid, well-structured Beamer code, including the correct syntax for frames, titles, and images.

5.9 Document Generation & Email

The system successfully generates `.pdf` (via `compile_latex_to_pdf`) and `.docx` (via `create_docx_report`) files. The email automation via `send_email` is fully functional. The only missing piece from the original brief's suggestion is a tool for `.pptx` *generation* (though `.pptx` *ingestion* is supported).

5.10 System Architecture & APIs

The codebase is highly modular and clean. The `api/main2.py` file provides a robust FastAPI server. The API is well-designed for the HITL workflow, with distinct, non-streaming endpoints for `/api/chat` and `/api/chat/resume`.

5.11 Final Deliverables & Demo

The GitHub repository contains all code. This document serves as the detailed report. The `api/main2.py` + `api/index2.html` files provide a fully functional, end-to-end interactive demo.

5.12 Bonus

Addressed. The project successfully implements multiple bonus features:

- **Reranking Step:** Implemented in `rag/rag_tools.py`.
- **Guardrails:** Implemented in `core/agent_self2.py` via `PIIMiddleware` and the custom `ContentFilterMiddleware`.
- **Advanced Agent Components:** The `HumanInTheLoopMiddleware` is an advanced workflow component that provides a robust verification and recovery layer.

6 Conclusion and Future Work

The Cosmic Compass project successfully delivers a powerful, agentic RAG system that meets all core project requirements and multiple bonus objectives. The system's true strength lies in its modular architecture and its robust, interruptible workflow, which combines autonomous agent execution with critical human oversight.

Future work could extend this project in several key areas:

- **Persistent Checkpointer:** Replace `InMemorySaver` with a persistent database like `AsyncPostgresSaver` to allow chat threads to survive server restarts.
- **Add PPTX Generation:** Implement a `create_pptx_report` tool using `python-pptx` to fully round out the document generation capabilities.
- **Advanced Error Recovery:** Enhance the supervisor's prompt to not just *stop* on failure, but to *retry* with a different format (e.g., "If PDF generation fails, try DOCX instead").

7 Demo Video

<https://drive.google.com/file/d/1E4oGmzJFmuBC8XkyFSf1ZruJytPwInON/view?usp=sharing>